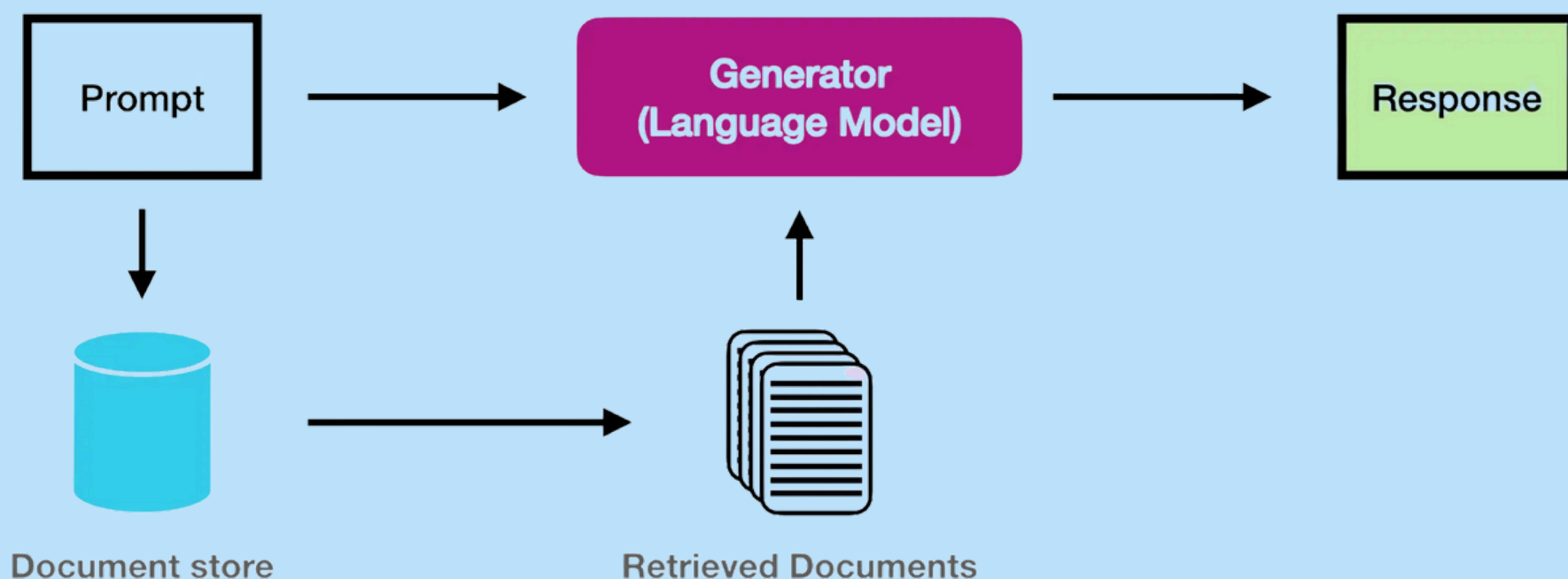


# RAG

## DICTIONARY

Your **A-to-Z** Guide to Mastering RAG!



**Chandra Sekhar**  
AI Engineer & Educator

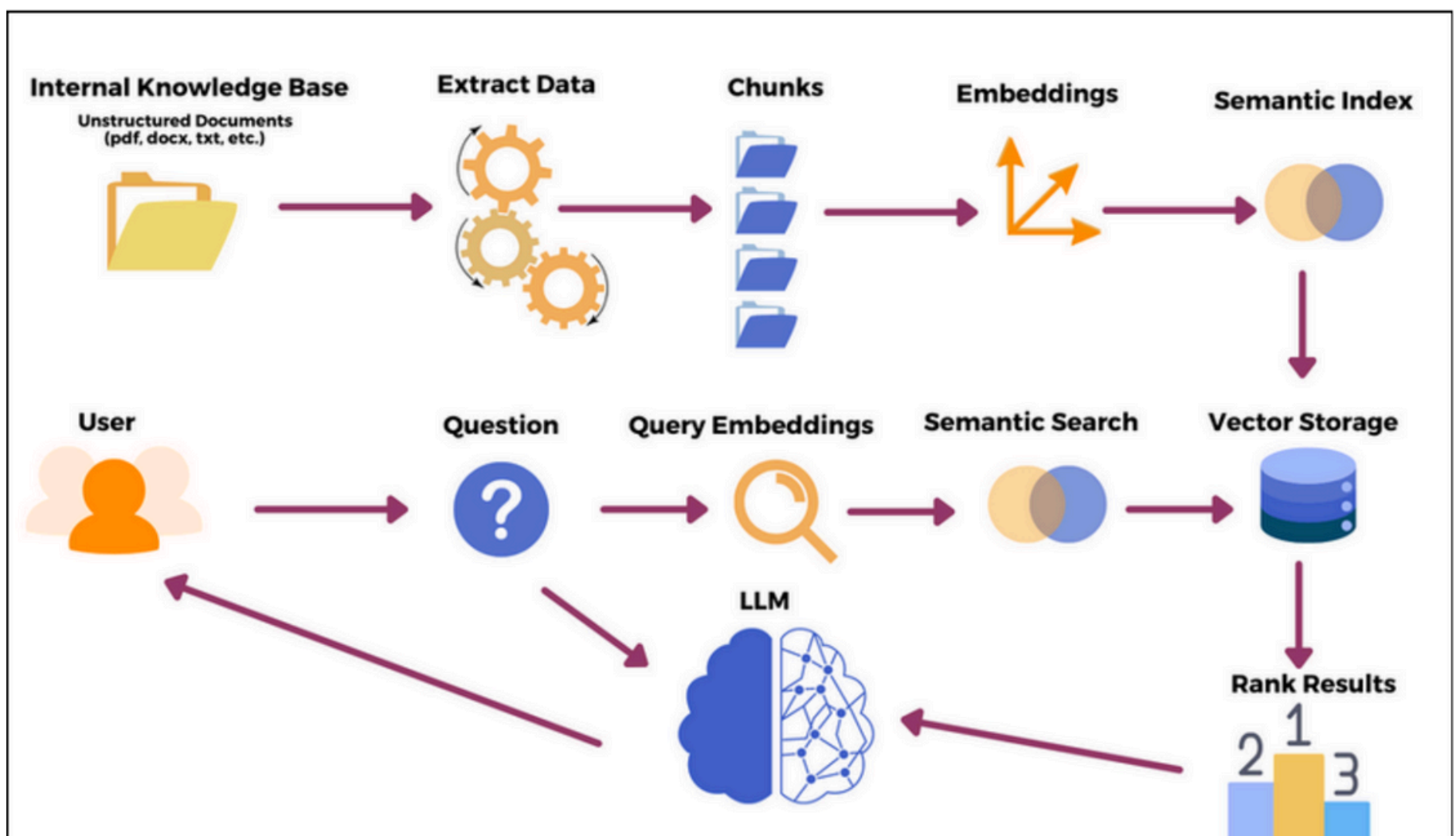




# Augmentation

Enhances generative models by incorporating external knowledge.

- Ensures outputs are factually accurate and contextually rich
- Reduces hallucinations in AI responses
- Widely used in customer support and document analysis systems





# BM25

A traditional ranking algorithm for scoring document relevance.

- Operates on the frequency and distribution of query terms in documents
- Ideal for keyword-based sparse retrieval tasks
- Forms the baseline for many RAG pipelines

$$BM25 = \sum_{t \in q} \log \left[ \frac{N}{df(t)} \right] \cdot \frac{(k_1 + 1) \cdot tf(t, d)}{k_1 \cdot \left[ (1 - b) + b \cdot \frac{dl(d)}{dl_{avg}} \right] + tf(t, d)}$$

- $k_1, b$  – parameters
- $dl(d)$  – length of document  $d$
- $dl_{avg}$  – average document length

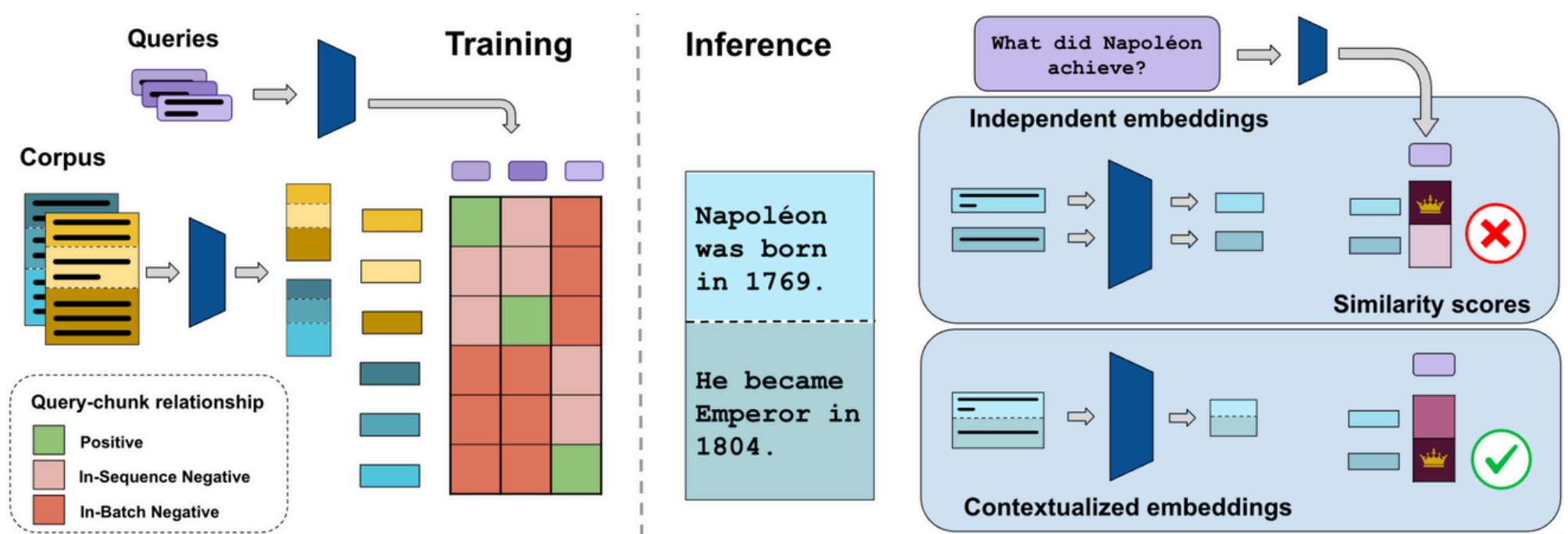




# Contextual Embedding

Represents words based on their surrounding text.

- Captures nuances and relationships in a sentence or document
- Key for aligning retrieved documents with queries
- Used in models like BERT to improve relevance



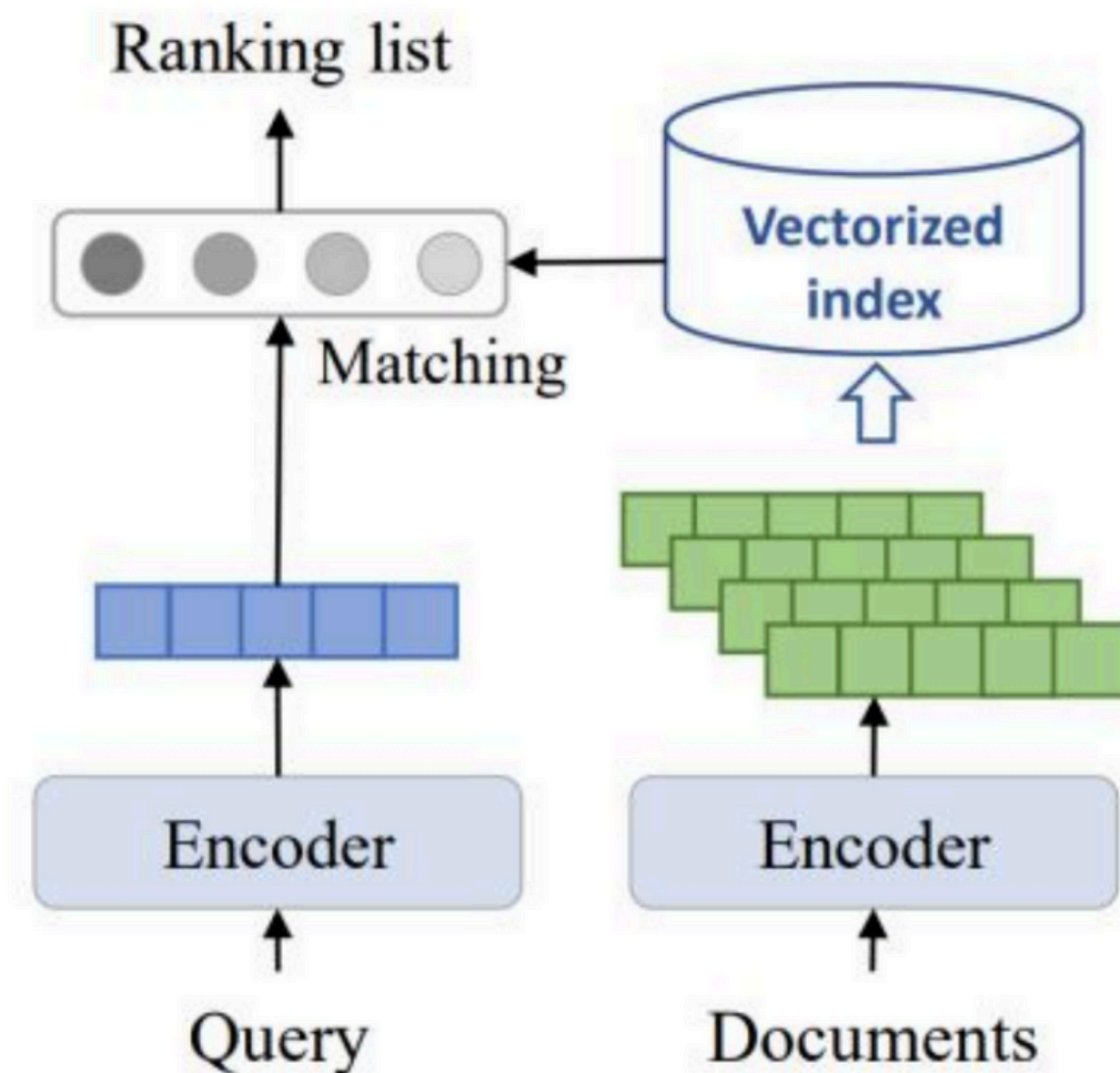




# Dense Retrieval

Uses embeddings to match queries with semantically similar documents.

- Powered by neural networks for high accuracy
- Excels at capturing contextual relevance
- Often paired with sparse retrieval for hybrid approaches





# Embeddings

Converts text or queries into fixed-size numerical vectors.

- Essential for comparing similarities in latent space
- Enables dense retrieval and semantic search
- Used in tasks like recommendation systems and RAG

data object



embedding model



data as vector embedding

|       |       |       |     |       |
|-------|-------|-------|-----|-------|
| 0.056 | 0.009 | -0.07 | ... | 0.023 |
|-------|-------|-------|-----|-------|

|       |       |       |     |       |
|-------|-------|-------|-----|-------|
| -0.06 | 0.039 | 0.137 | ... | 0.045 |
|-------|-------|-------|-----|-------|

|       |       |       |     |       |
|-------|-------|-------|-----|-------|
| 0.012 | -0.01 | 0.079 | ... | -0.02 |
|-------|-------|-------|-----|-------|

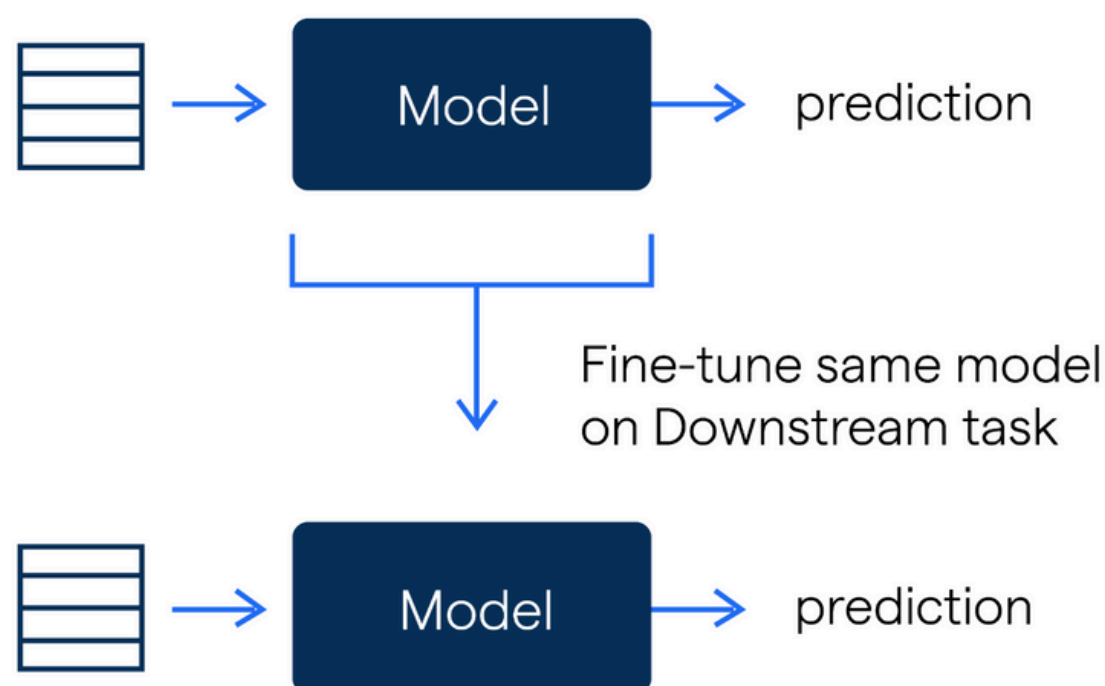




# Fine-tuning

Customizes pre-trained models for domain-specific tasks.

- Aligns retrieval and generative models for better performance
- Increases accuracy for industry-specific use cases
- Requires labeled data for optimal results

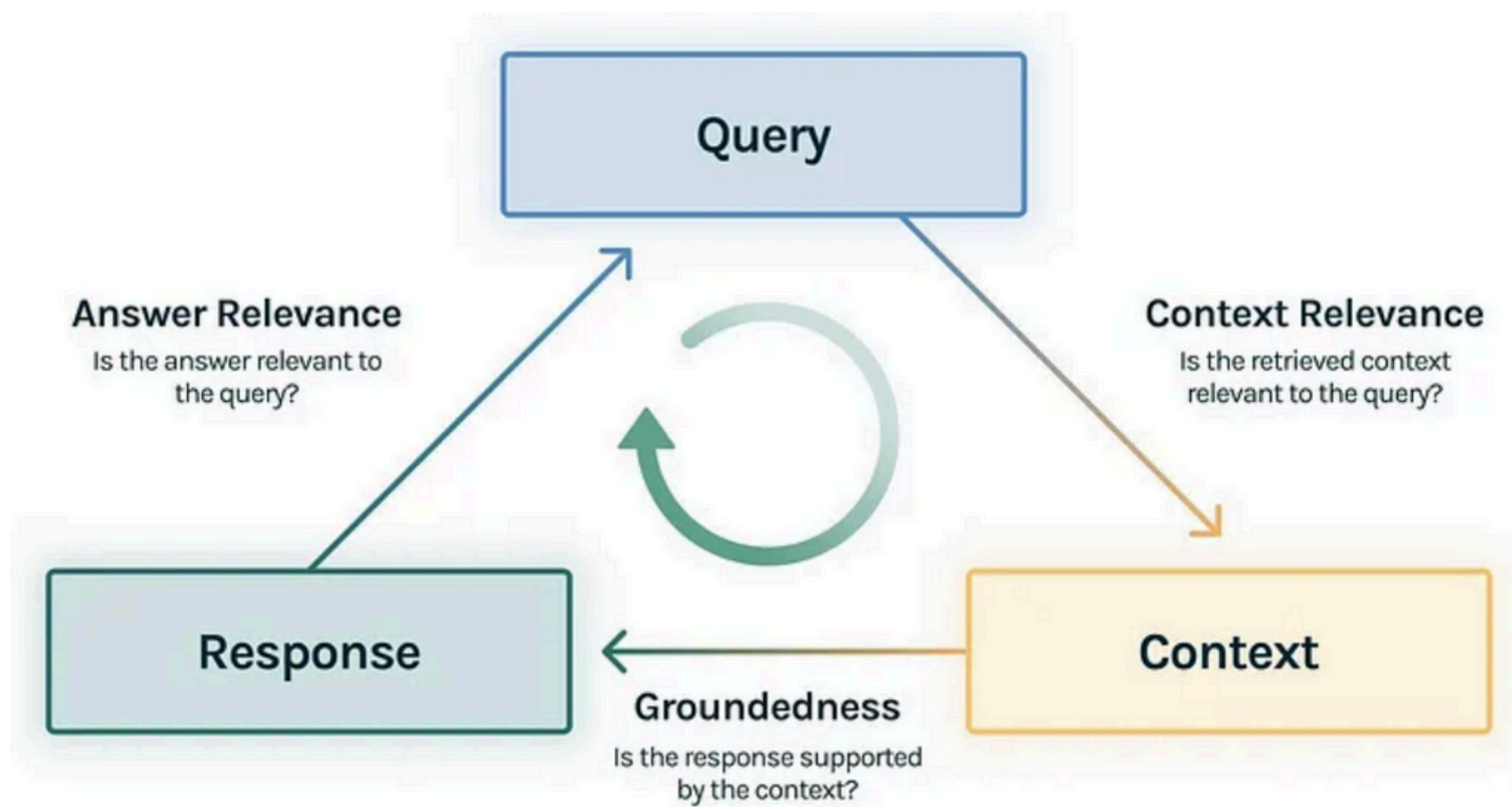




# Grounding

Ensures that generated outputs are supported by retrieved knowledge.

- Helps models stay factually consistent and trustworthy
- Critical in applications like medical AI and legal tech
- Mitigates the risk of hallucinated or false information



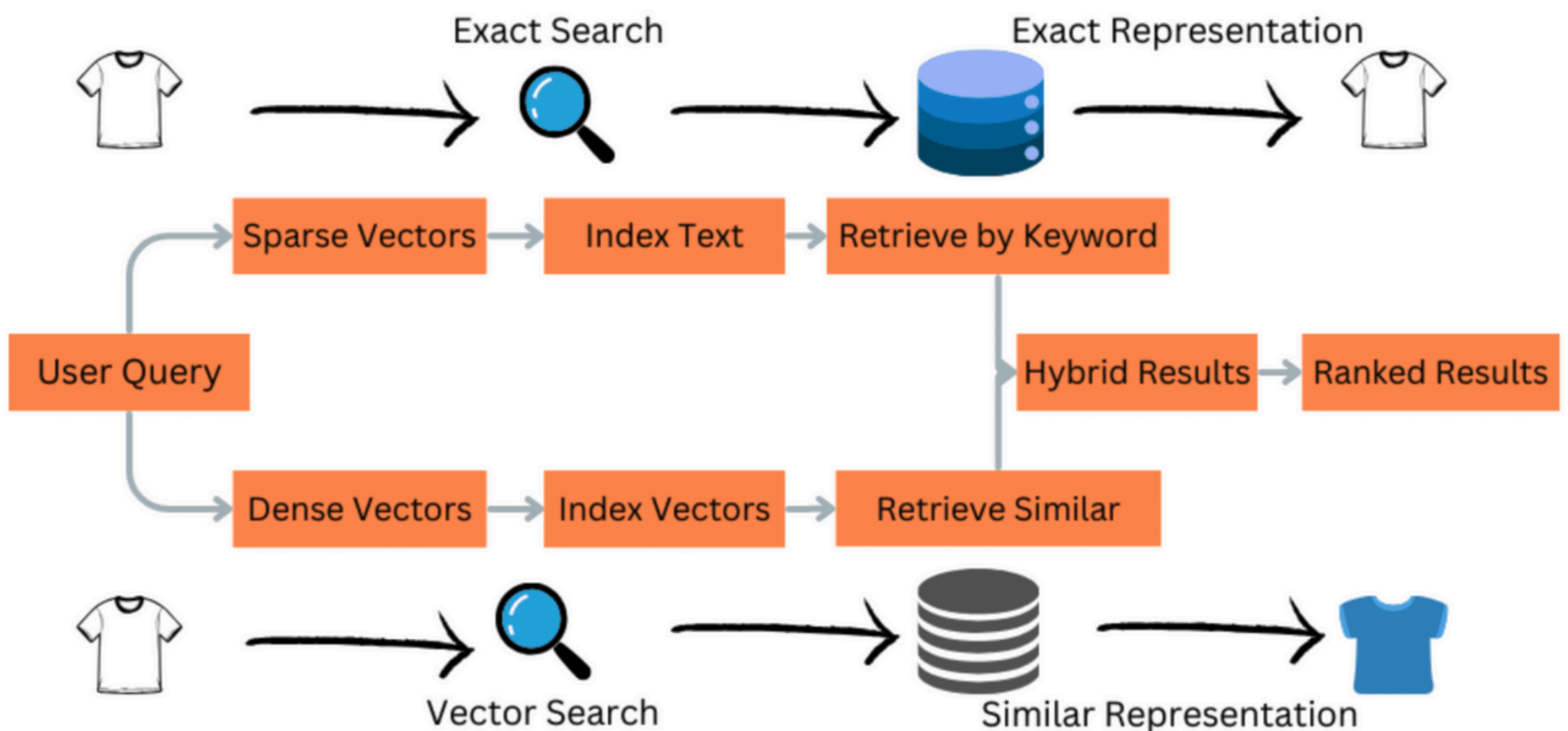




# Hybrid Search

Combines sparse (e.g., BM25) and dense (embedding-based) methods.

- Balances efficiency and accuracy for document retrieval
- Reduces reliance on a single retrieval approach
- Common in real-world RAG systems for scalability

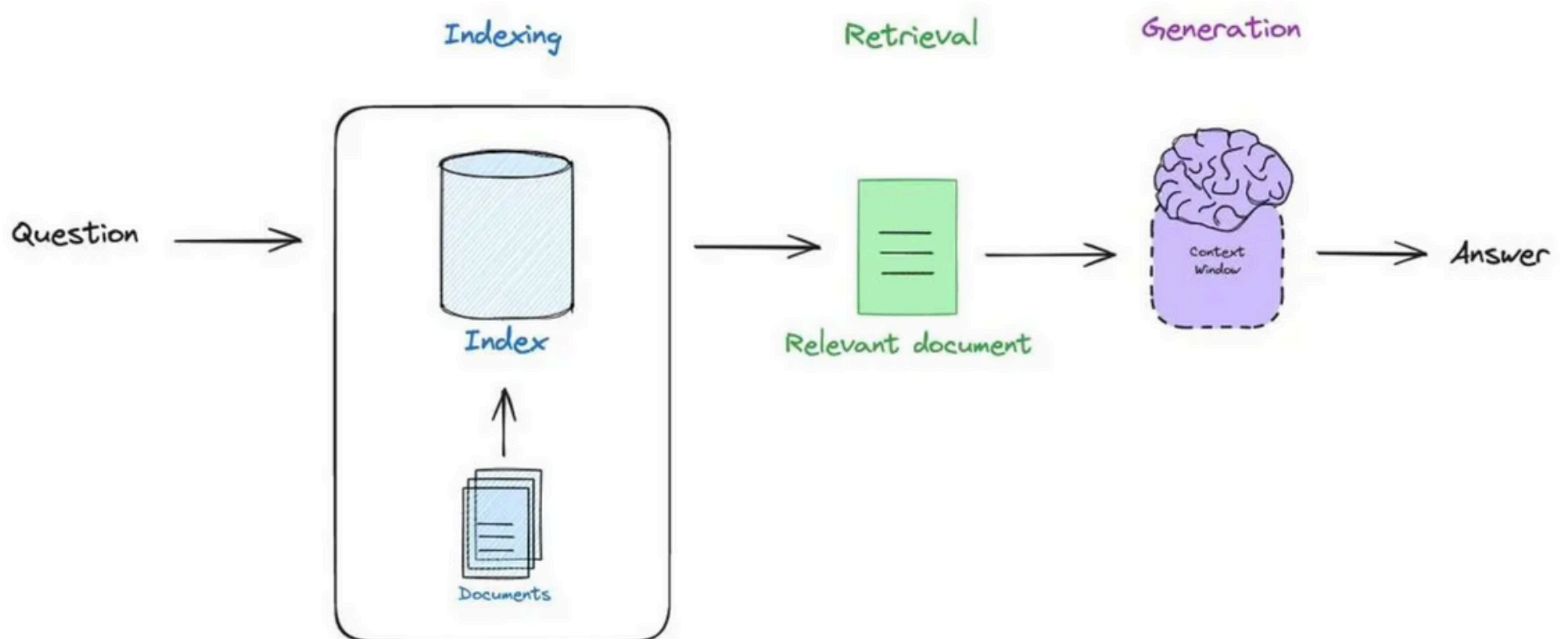




# Indexing

Organizes data for quick and efficient retrieval.

- Structures documents into formats compatible with retrieval systems
- Reduces query latency in RAG pipelines
- Supports scalable search across large datasets

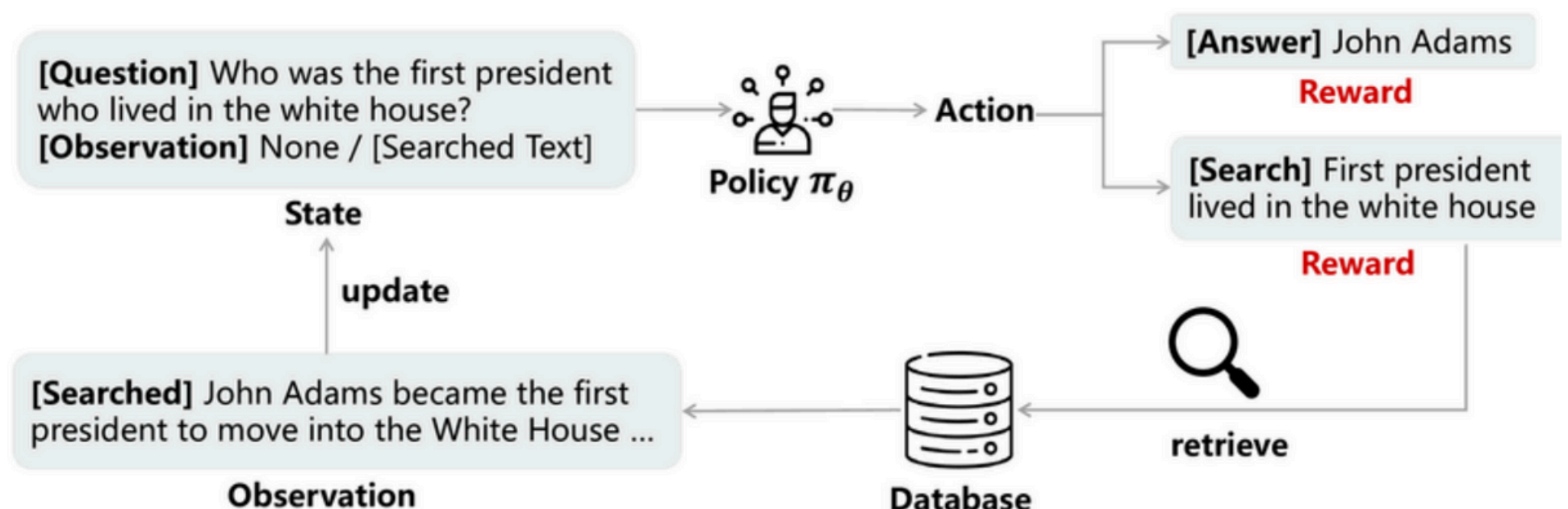




# Joint Learning

Trains retrieval and generation components simultaneously.

- Ensures better synergy between retrieved data and generated outputs
- Reduces the need for separate fine-tuning
- Often leads to improved overall system performance

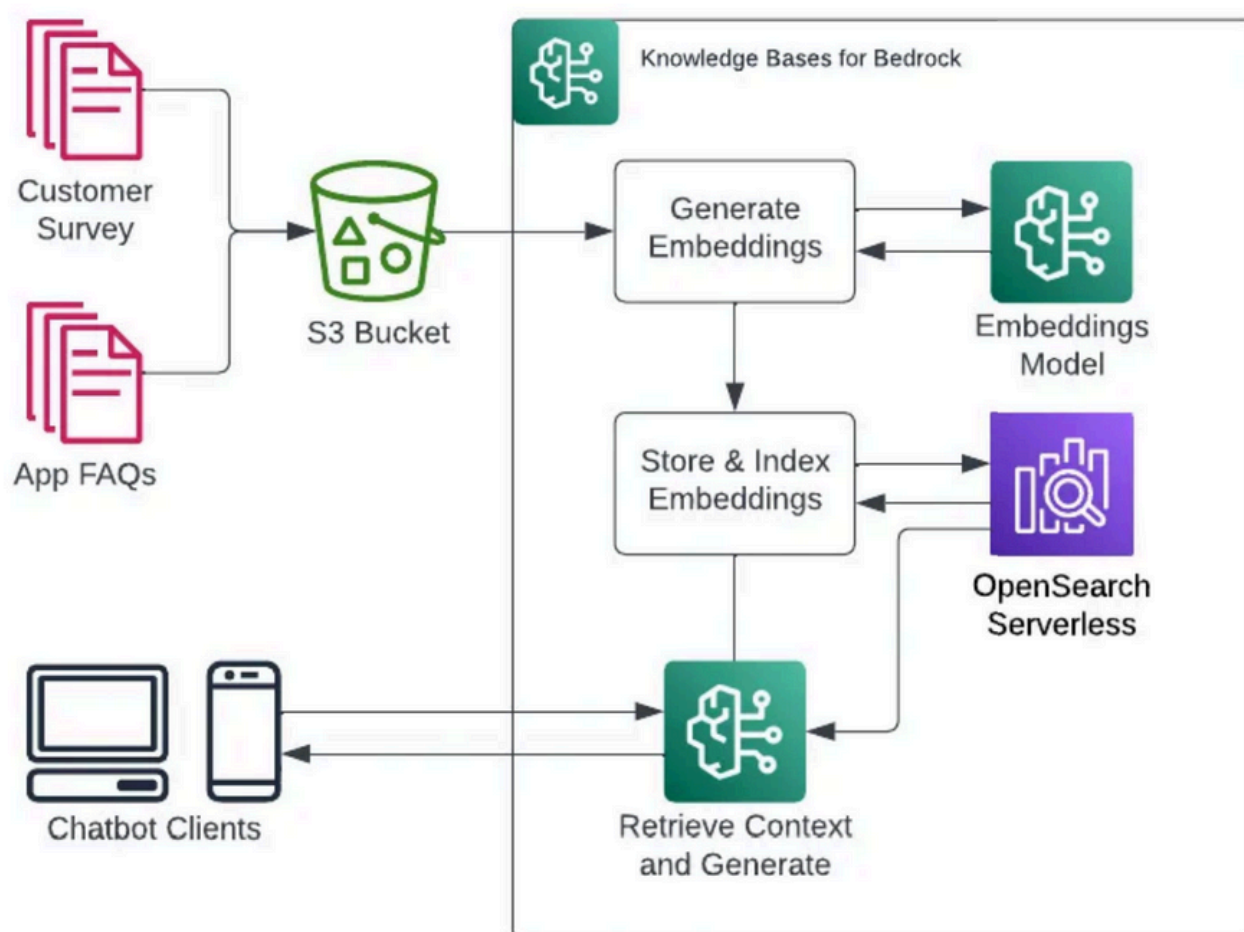




# Knowledge Base

A structured repository of data for retrieval purposes.

- Includes formats like databases, ontologies, and indexed documents
- Powers fact-checking and real-time information retrieval in RAG
- Ensures reliability and accuracy in responses



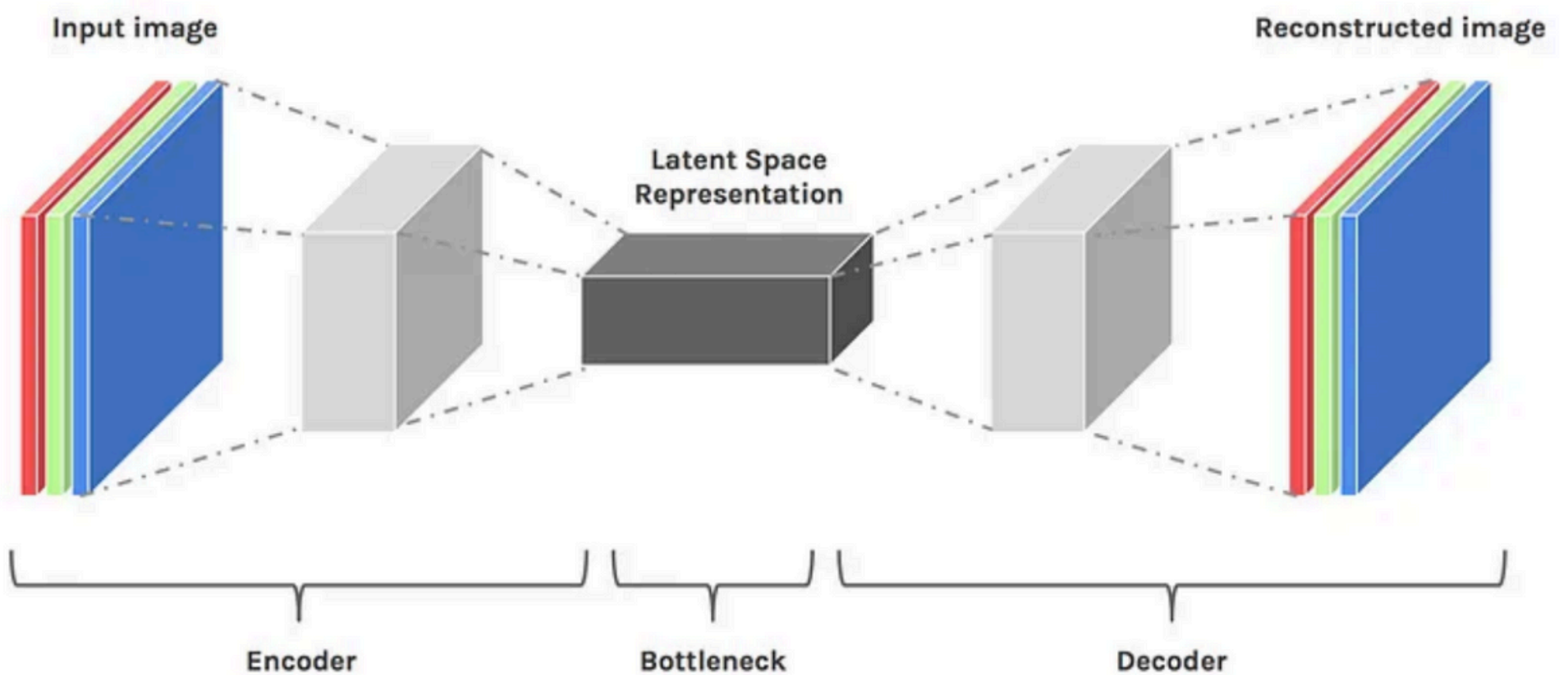




# Latent Space

A high-dimensional space where embeddings are mapped.

- Enables semantic matching between queries and documents
- Used for clustering similar concepts or topics
- Visualized to understand model behaviors and relationships

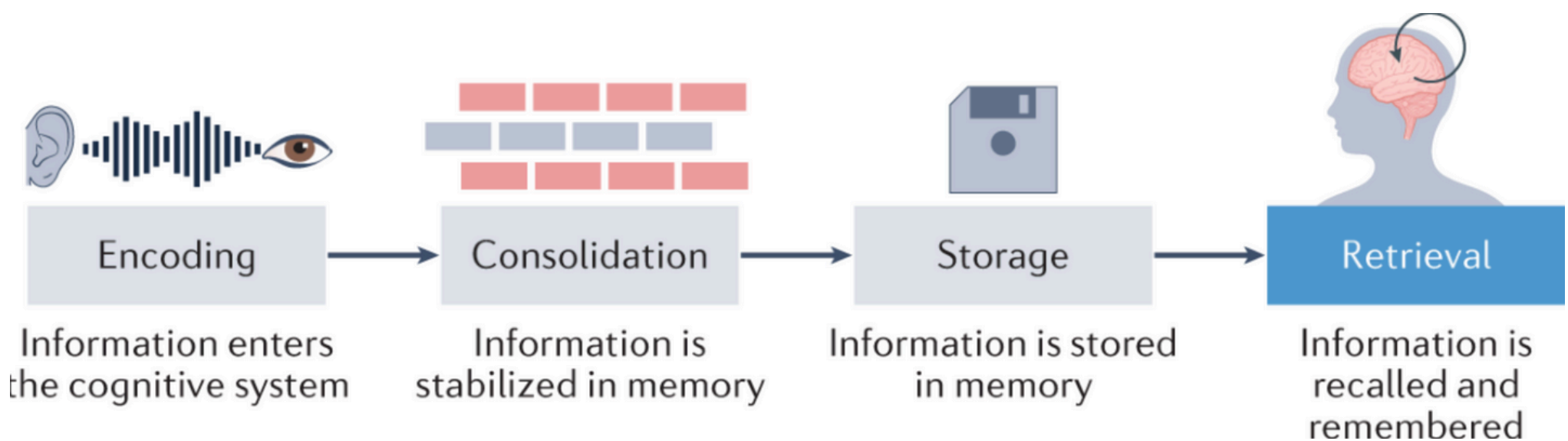




# Memory Retrieval

Focuses on fetching historical data or past interactions.

- Helps personalize user experiences
- Common in chatbots and recommendation systems
- A subset of retrieval tailored to specific past contexts

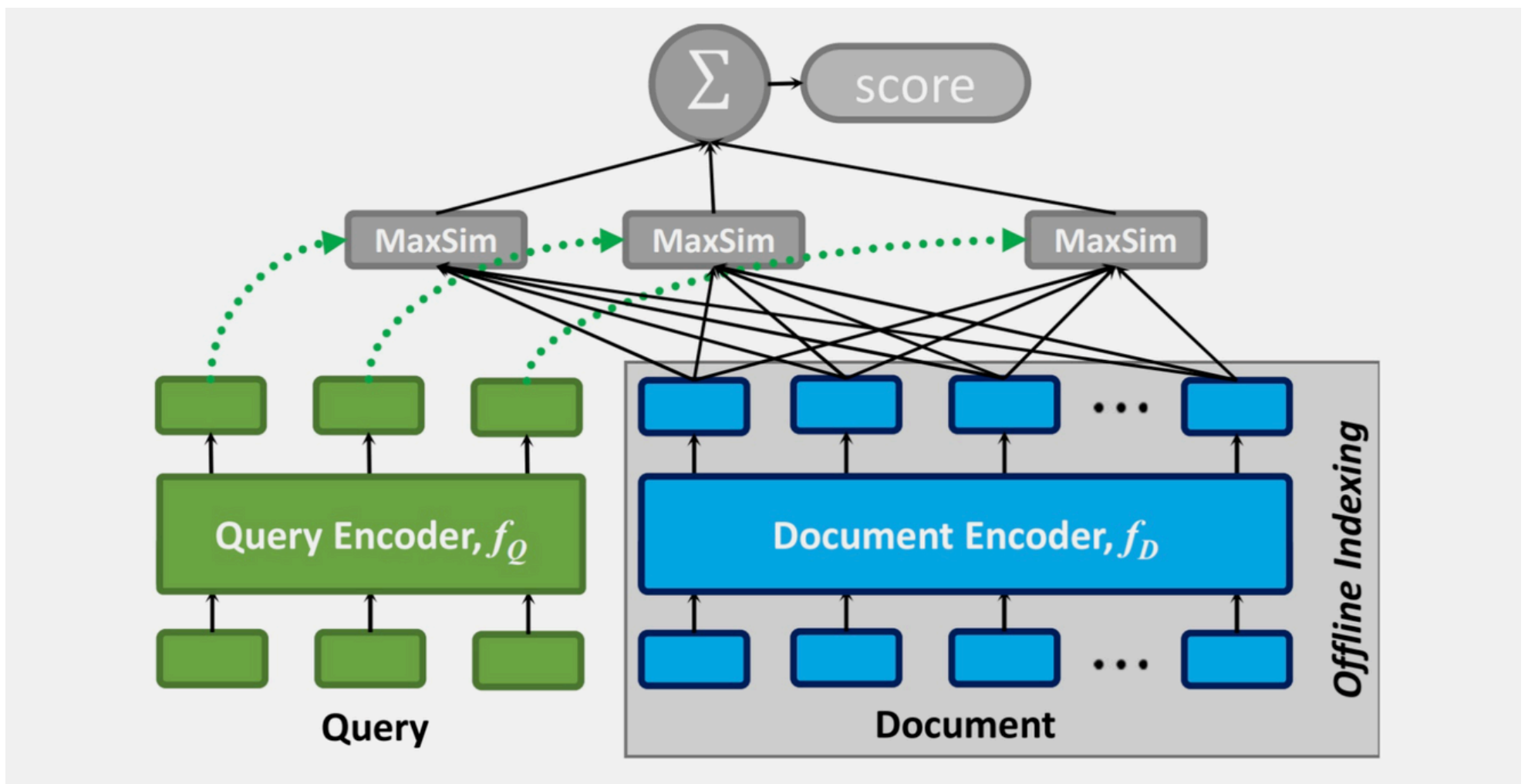




# Neural Retrieval

Leverages deep learning for document-query matching.

- Captures semantics beyond exact keyword matches
- Includes models like DPR (Dense Passage Retrieval)
- Enhances relevance in large-scale document search tasks

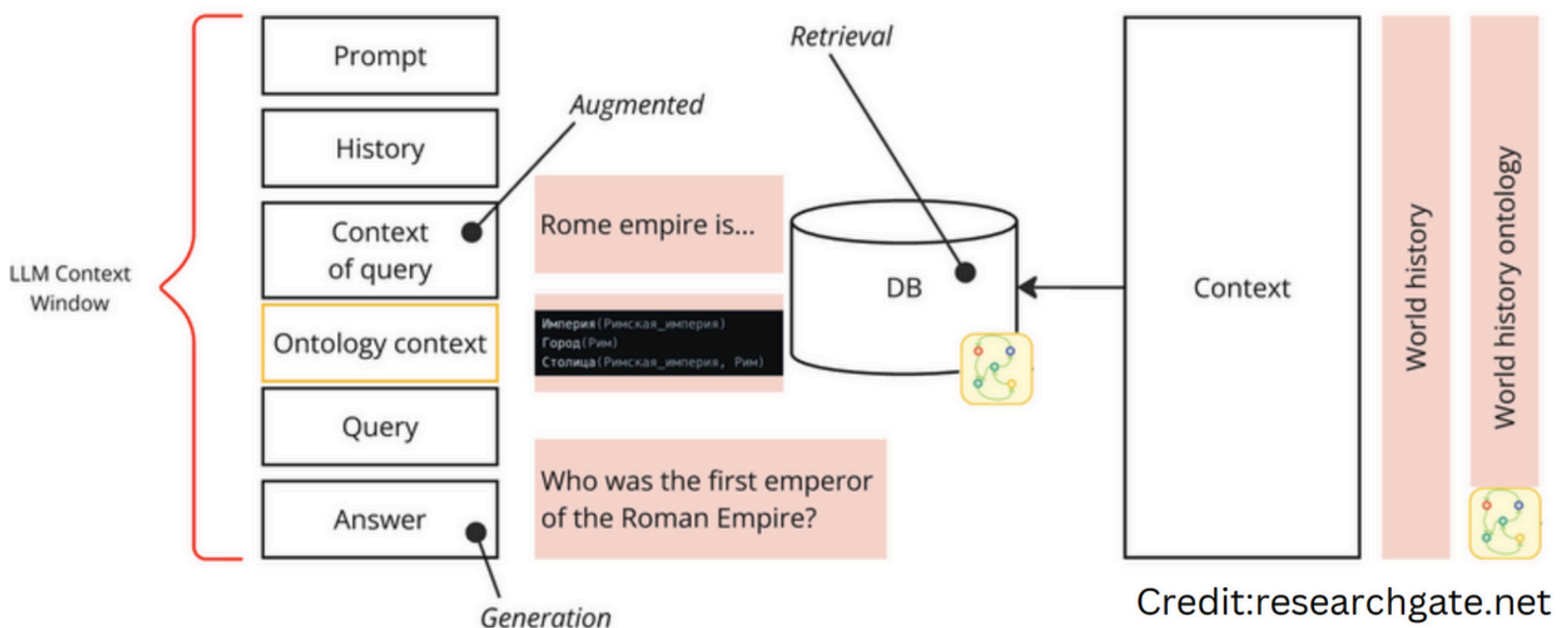




# Ontology

Represents structured relationships between entities.

- Used in knowledge representation and retrieval
- Enables semantic understanding of complex concepts
- Supports domain-specific search and reasoning



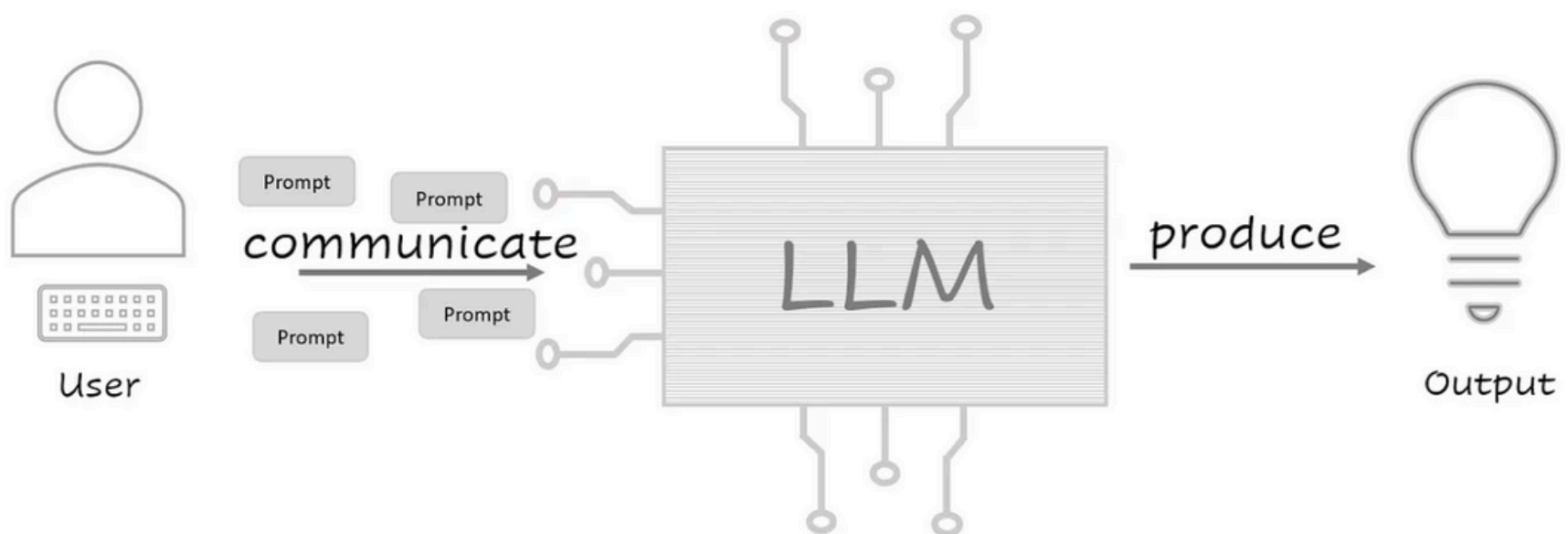




# Prompt Engineering

Designs prompts to guide generative models effectively.

- Ensures retrieved knowledge is integrated into outputs
- Essential for achieving task-specific responses
- Iterative and domain-specific in application

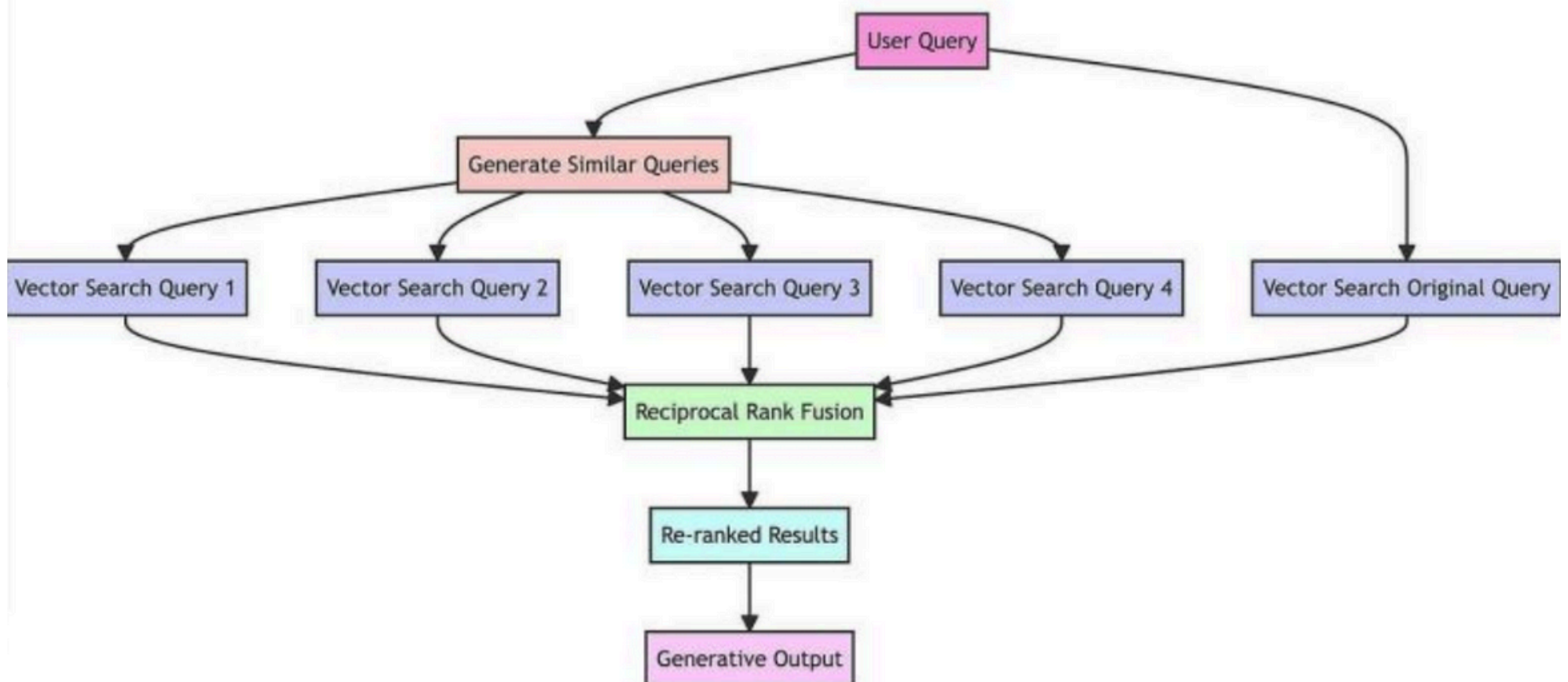




# Query Expansion

Broadens the scope of search by adding related terms or synonyms.

- Improves recall in sparse and dense retrieval systems
- Mitigates the issue of ambiguous or short queries
- Often applied in user-facing search systems

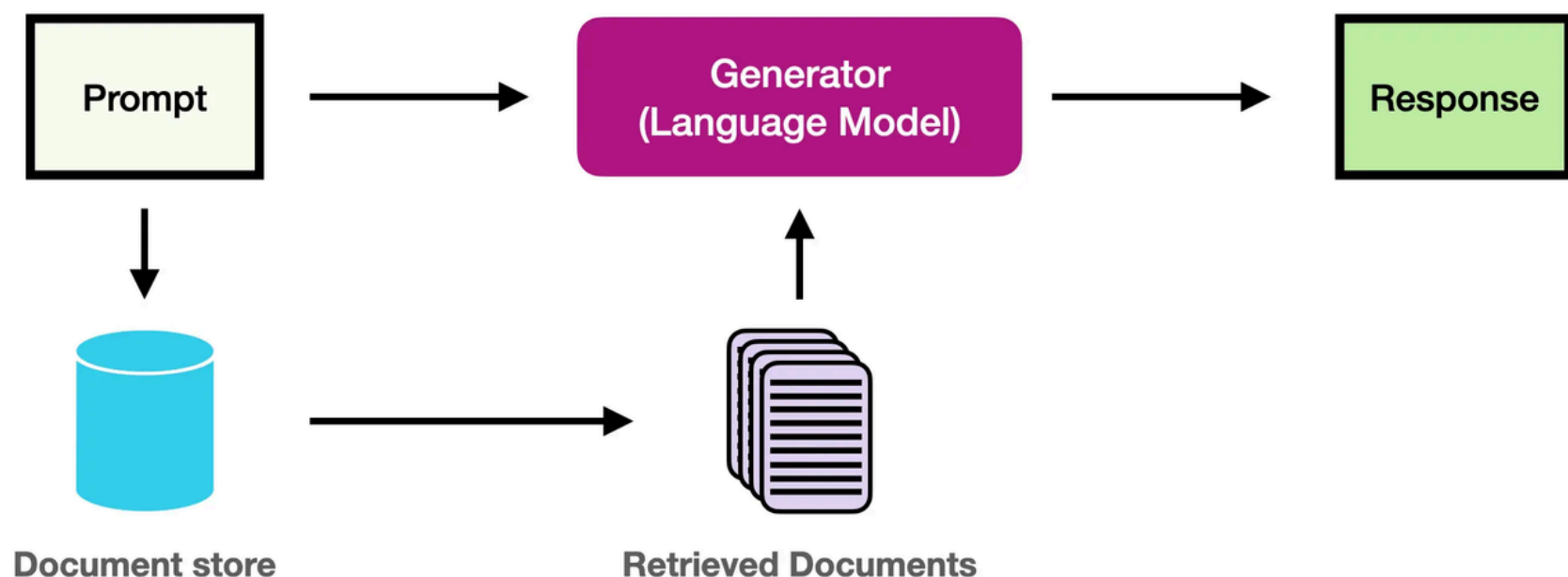




# Retrieval-Augmented Generation (RAG)

Combines retrieval systems with generative models for contextual outputs.

- Minimizes hallucinations by grounding outputs in retrieved data
- Widely used in knowledge-intensive tasks
- Represents a hybrid approach to solving information and generation problems

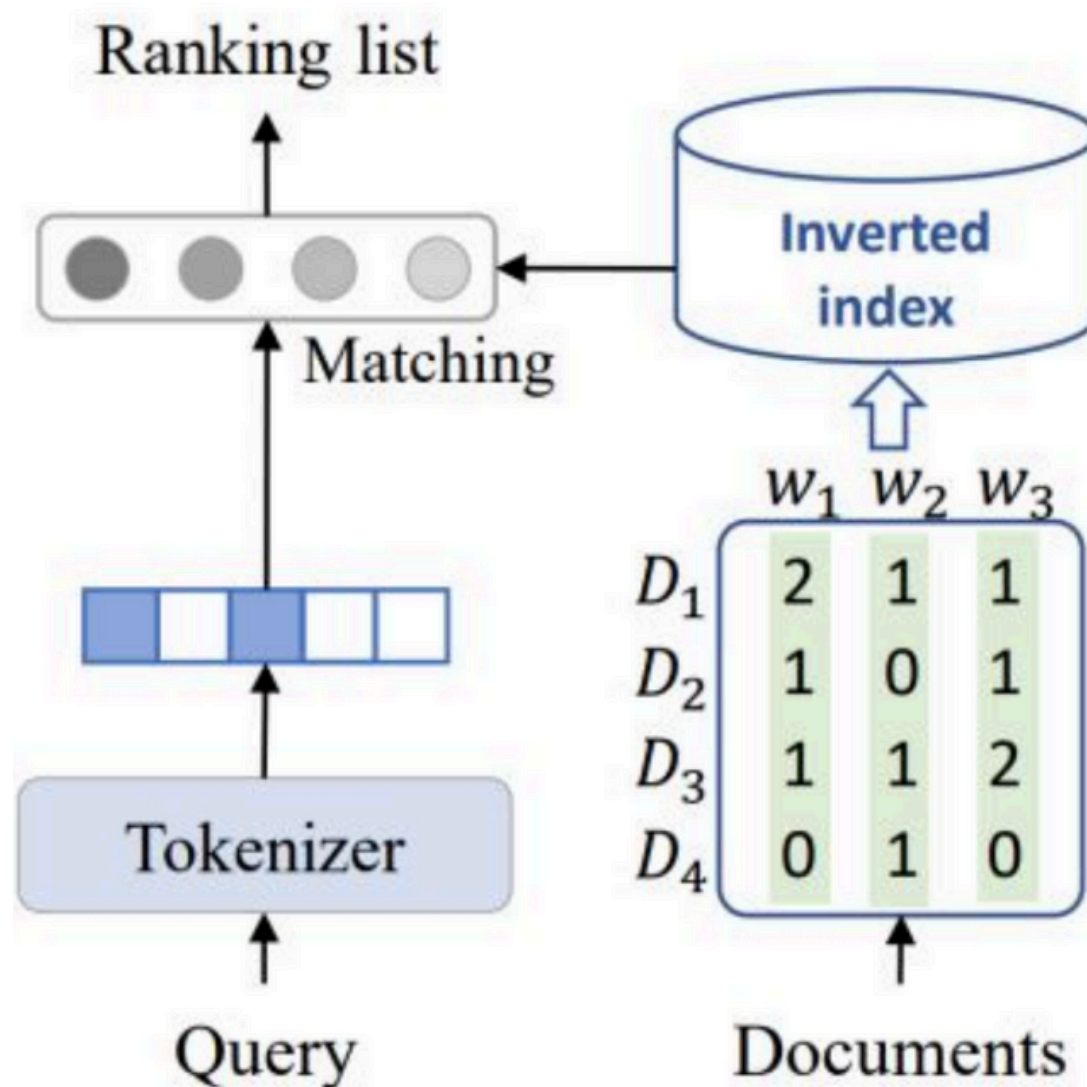




# Sparse Retrieval

Relies on keyword-based methods like BM25 and TF-IDF.

- Efficient and interpretable for small-scale datasets
- Struggles with semantic or contextual understanding
- Often augmented with dense retrieval for hybrid systems





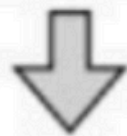


# Tokenization

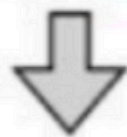
Splits text into smaller units (tokens) for processing.

- Necessary for model compatibility and efficient computation
- Handles text variations like punctuation and capitalization
- Used in retrieval and generation pipelines

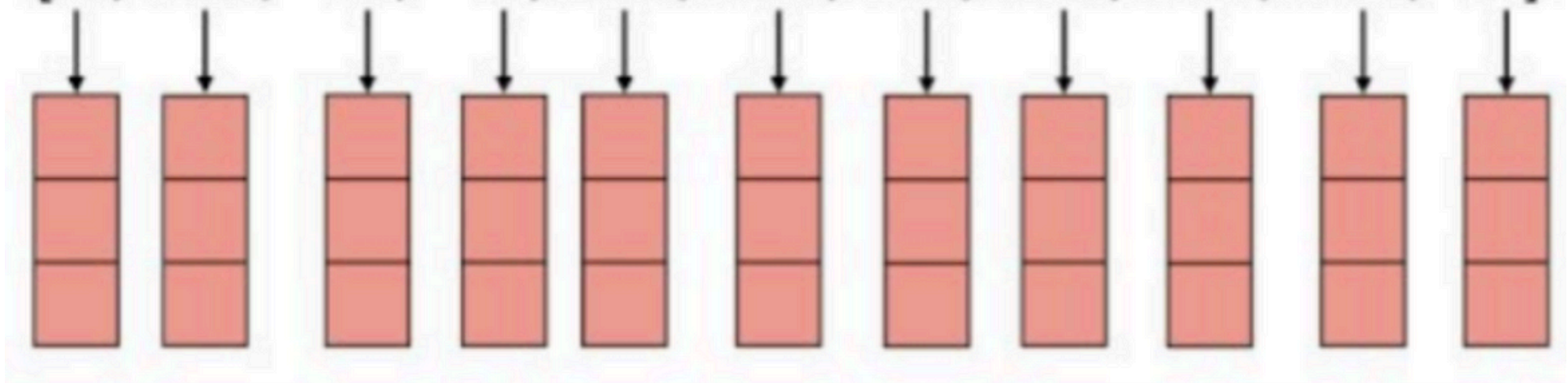
"Today is a beautiful day outside."



["To", "day", "is", "a", "beaut", "iful", "day", "out", "side", "."]



[98, 1452, 43, 15, 2932, 1709, 740, 1452, 3112, 3823, 74]

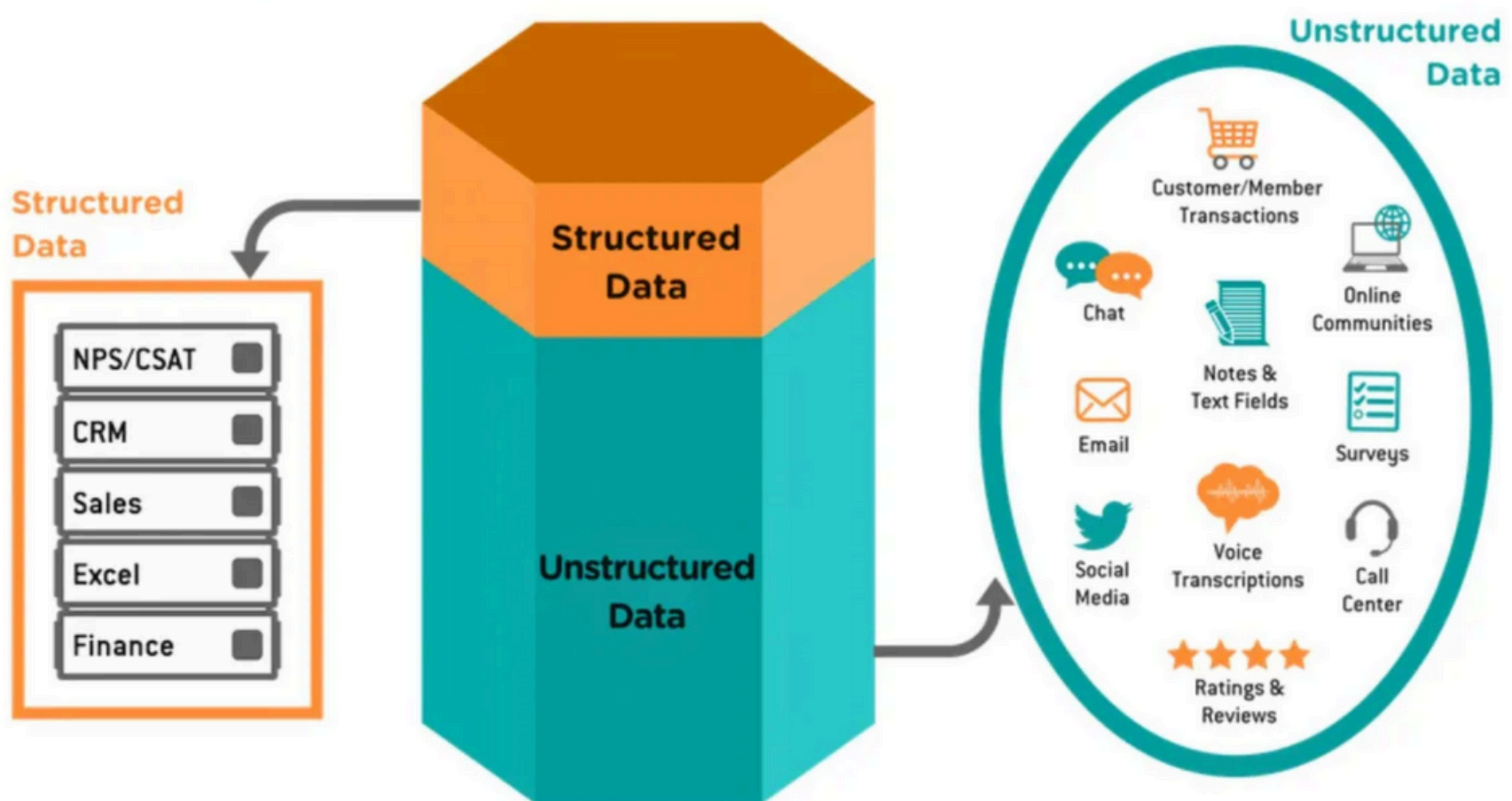




# Unstructured Data

Raw data formats like text, images, or videos.

- Requires preprocessing for effective retrieval and generation
- A significant challenge in RAG systems
- Powers applications like multimedia search and document summarization

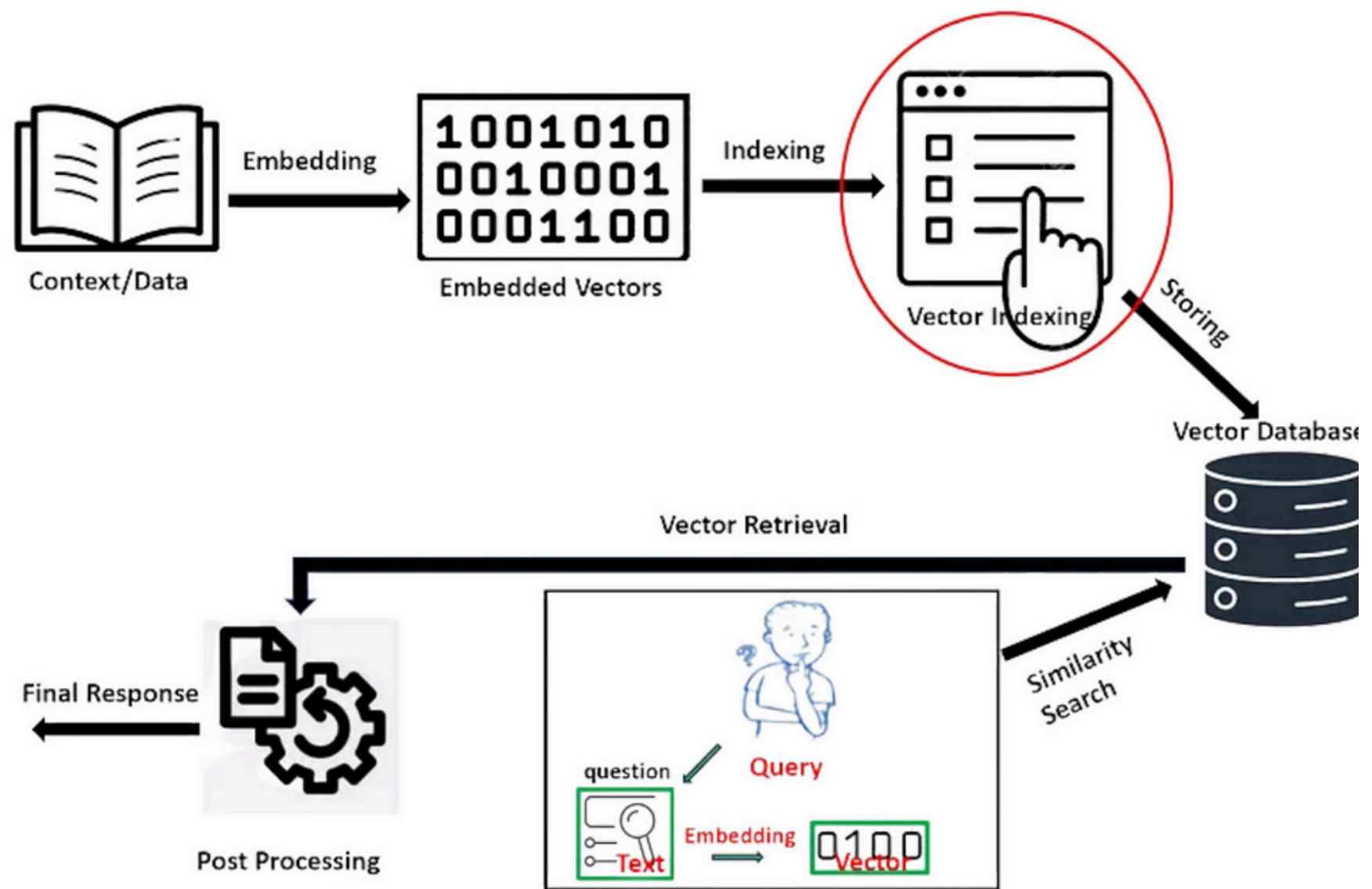




# Vector Search

Uses embeddings to find semantically similar items.

- Operates in high-dimensional latent space
- A core method for dense retrieval in RAG systems
- Supports scalability in real-time search applications



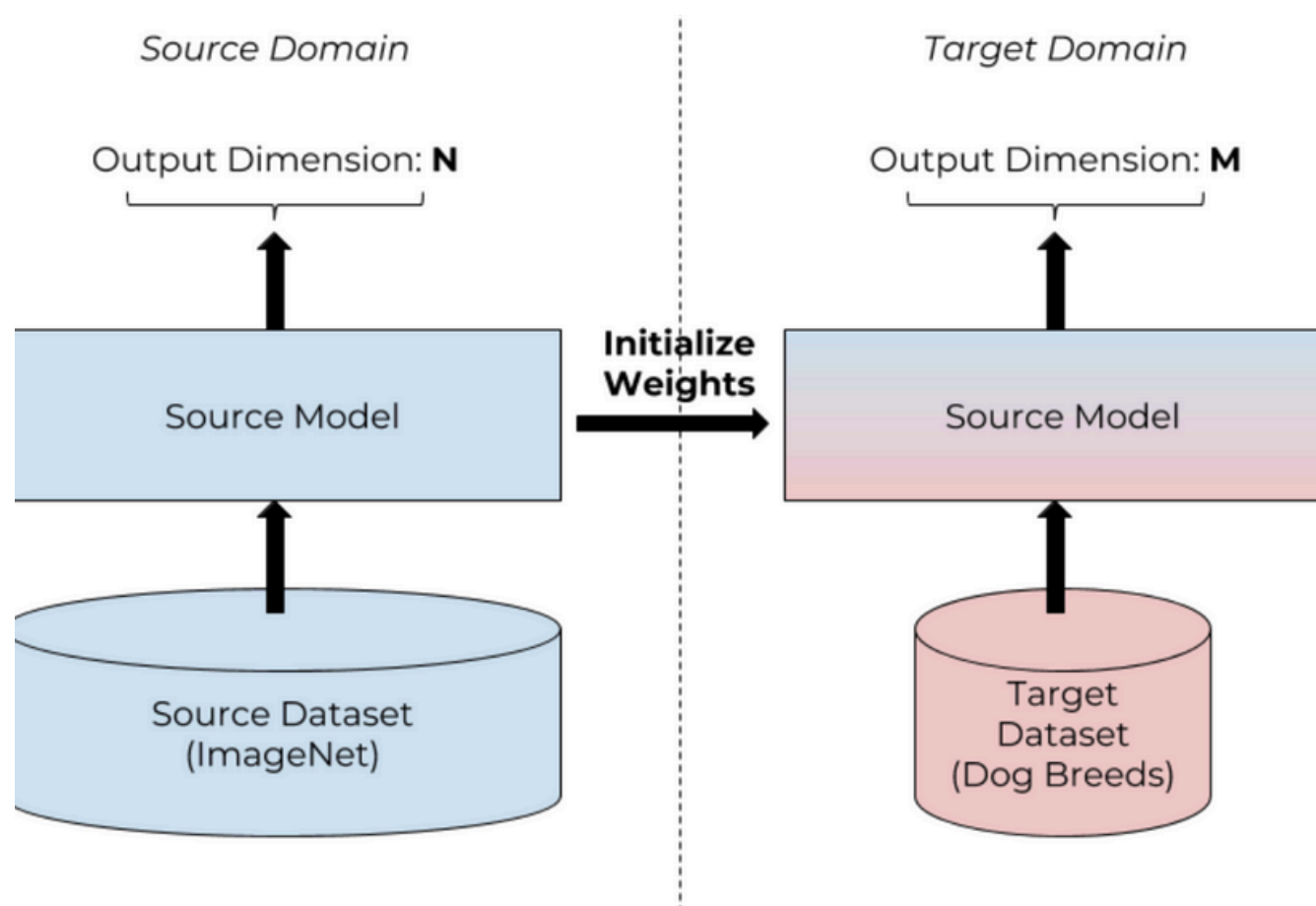




# Warm-Start Retrieval

Initializes retrieval systems with pre-trained embeddings or models.

- Speeds up convergence and improves early-stage performance
- Reduces training time for new tasks
- Common in transfer learning scenarios



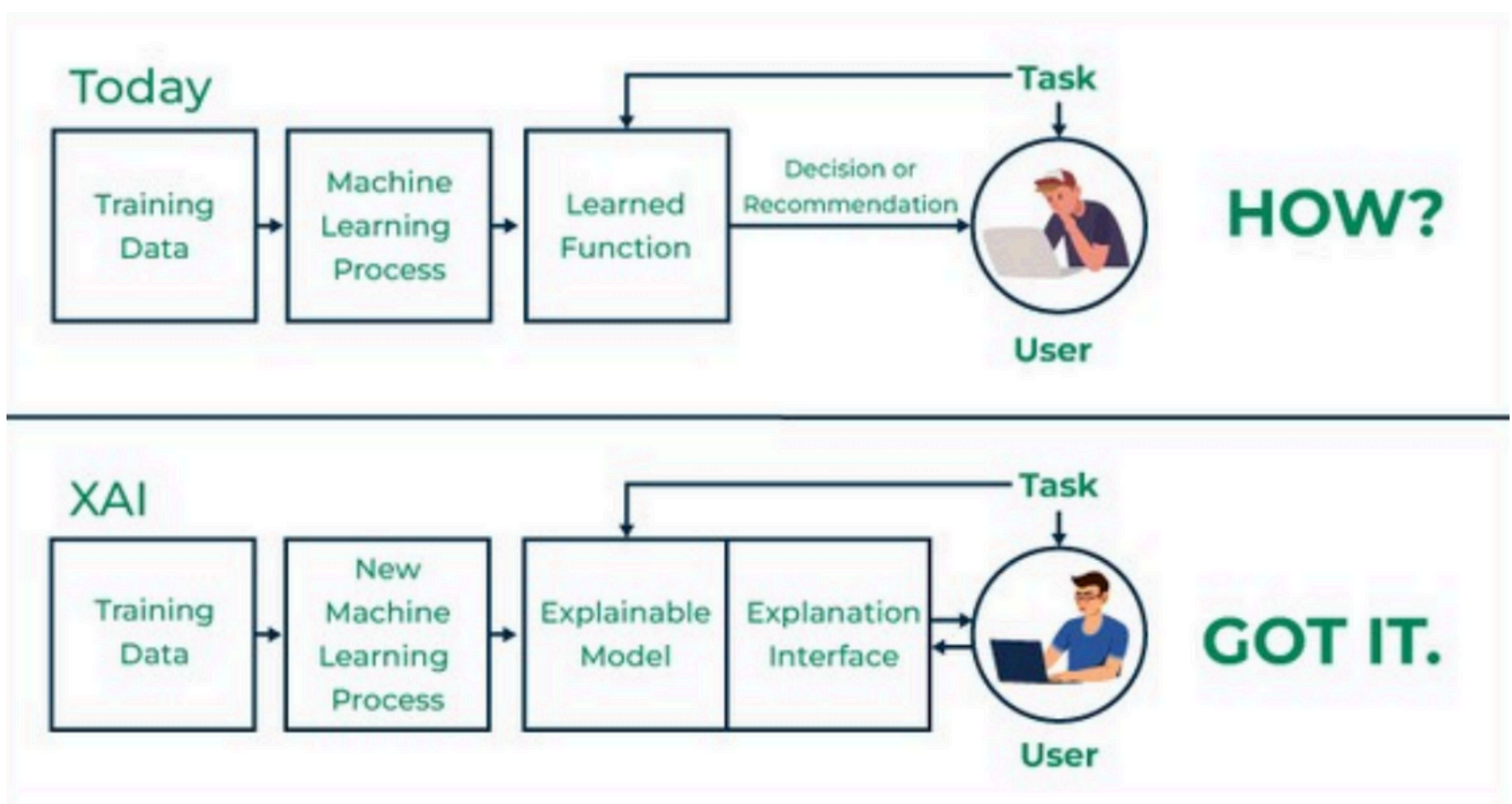




# Explainability

Traces how retrieved documents contribute to generated outputs.

- Ensures transparency and accountability in AI systems
- Critical for high-stakes applications like healthcare or law
- Helps identify and debug errors in RAG pipelines

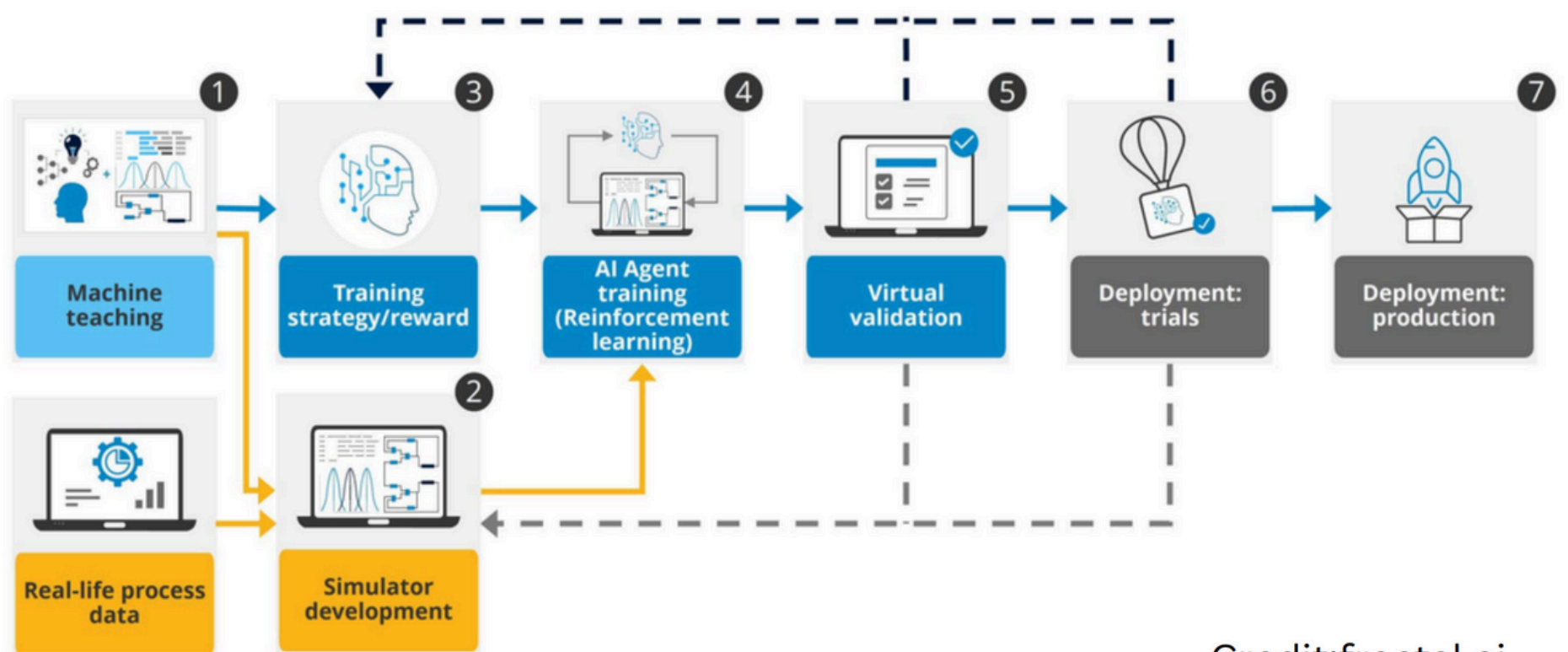




# Yield Optimization

Maximizes the relevance and quality of retrieved documents.

- Focuses on optimizing generative responses based on retrieval
- Involves fine-tuning retrieval components for performance
- Enhances user satisfaction in RAG-powered applications



Credit:fractal.ai





# Zero-shot Retrieval

---

Retrieves relevant information without prior task-specific training.

- Useful in scenarios with minimal or no labeled data
- Relies heavily on pre-trained models
- Key for adapting to new domains or tasks quickly



# Want more content like this?



**Tap that follow button and  
stay in the loop!**



**Like**



**Comment**



**Share**



**Save**